

Project Report

Team 73: Replicated Fault Tolerant Key Value Store

MONISH (2022111003); RISHABH (2023121001)

November 28, 2024

Contents

1	What is Raft?	2
2	Our Implementation	2
2.1	Node Management	2
2.2	Raft Server Logic	3
2.3	RPC Handling	3
2.4	Key-Value Store	4
2.5	Snapshot and Log Compaction	4
2.6	Server Initialization	4
3	Benchmarks	5
3.1	Performance Metrics	5
3.2	Scalability Testing	5
3.3	Fault Tolerance Evaluation	5
4	Conclusion	5
5	Work Distribution	5
5.1	Common parts	5
5.2	Rishabh (2023121001)	5
5.3	Monish (2022111003)	6
6	Bibliography	7

1 What is Raft?

The project was an attempt at making a fault-tolerant key value store. To replicate the various nodes and ensure that they are consistent, we use Raft (Ongaro, Diego and Ousterhout, John, 2014), an alternative to Multi-Paxos (Lamport, Leslie, 1998) that is touted to be simpler to understand and implement. One key difference between Multi-Paxos and Raft is the strong leader property, which enforces the logs of the leader on all of the follower nodes to ensure that all nodes are synced. This ensures that there are no updates lost on the way.

Raft is not intended for fully replacing a key-value store, rather using it as a fundamental building block on top of other services (for example: lock services, configuration, etc.). This is because all updates for Raft have to pass through the leader, which creates a bottleneck for the cluster. Raft also errs towards consistency over availability, further impacting performance.

2 Our Implementation

We implemented Raft in Python, primarily due to the flexibility offered by it, as well as the familiarity. The bulk of the code involving the implementation of Raft is present in `raft_server.py`. The implementation consists of several key components that work together to provide a fault-tolerant key-value store.

2.1 Node Management

The `node.py` module defines the `Node` class, which represents a single Raft node. This class is responsible for handling HTTP requests via the `NodeHTTPRequestHandler` and managing communication between nodes. The node operates as a singleton to ensure that there is only one instance per process. It registers endpoints for handling various RPCs and manages message passing between peers using the `send_message` method.

Key functions and classes include:

- `NodeHTTPRequestHandler`: Handles incoming HTTP POST requests and delegates them to the appropriate RPC handlers.
- `SingletonMixin`: Ensures that only one instance of the `Node` class exists.
- `send_message`: Facilitates sending messages to peer nodes, including simulating message drops based on the `message_success_rate`.

2.2 Raft Server Logic

The core Raft logic is encapsulated within the `RaftServer` class in `raft_server.py`. This class manages the Raft consensus algorithm, including leader election, log replication, and state transitions. Key functions include:

- `start_election`: Initiates a new election by transitioning the server to the Candidate state, incrementing the current term, and soliciting votes from peers.
- `become_leader` and `become_follower`: Handle transitions between different server states based on the consensus protocol.
- `RequestVote` and `AppendEntries`: RPC handlers that process incoming vote requests and log replication messages from other nodes.
- `send_heartbeats`: Periodically sends heartbeat messages to followers to maintain leadership and prevent new elections.
- `apply_committed_entries` and `create_snapshot`: Manage the application of committed log entries to the state machine and handle log compaction through snapshotting.
- `start_election_timer` and `run_heartbeat_timer`: Manage timing mechanisms for elections and heartbeats.

2.3 RPC Handling

The `rpc.py` module provides a decorator-based approach to handling RPCs. The `rpc_call` decorator simplifies the registration and invocation of RPC methods by automatically serializing and deserializing messages. This ensures type-safe communication between nodes and abstracts away the underlying networking details.

Key functions and classes include:

- `rpc_call`: A decorator that registers functions as RPC handlers and manages serialization/deserialization of messages.
- `Hello`: An example RPC function demonstrating how RPCs are defined and used within the system.

2.4 Key-Value Store

Implemented in the `raft_server.py`, the `KeyValueStore` class provides a thread-safe key-value store that serves as the state machine for the Raft consensus algorithm. It includes `set` and `get` methods to handle client requests for modifying and retrieving data. This component ensures that all updates are consistent and properly logged for replication across the cluster.

Key functions include:

- `set`: Adds or updates a key-value pair in the store.
- `get`: Retrieves the value associated with a given key.

2.5 Snapshot and Log Compaction

To prevent the log from growing indefinitely, the implementation includes log compaction through snapshots. The `Snapshot` class represents a snapshot of the state machine, and methods like `create_snapshot` and `apply_snapshot` manage the creation and application of these snapshots. This ensures that the system remains efficient and does not consume excessive memory over time.

Key functions include:

- `create_snapshot`: Serializes the current state of the key-value store and truncates the log up to the snapshot point.
- `apply_snapshot`: Applies a received snapshot to the local state machine, ensuring consistency across nodes.
- `verify_snapshot.py`: A utility script to verify the integrity and correctness of snapshots.

2.6 Server Initialization

The `server.py` script serves as the entry point for starting a Raft node. It parses command-line arguments to configure the node ID, port, peers, and other settings. It initializes the `Node` and `RaftServer` instances and starts the node's HTTP server to listen for incoming RPCs. Additionally, it provides a command-line interface for interacting with the node, allowing users to send commands such as setting and getting key-value pairs or manually promoting a node to leader.

Key functions include:

- **main:** Parses arguments, initializes logging, sets up the node and Raft server, and manages the command-line interface for user interactions.

3 Benchmarks

3.1 Performance Metrics

3.2 Scalability Testing

3.3 Fault Tolerance Evaluation

4 Conclusion

While it is an easy algorithm to understand and implement, it is not a replacement for the Paxos protocol. There are several improvements that Paxos has seen that can be translated to a modified protocol for Raft.

However, Raft is successful in its own right, due to its simplicity and orthogonality of concerns, being widely deployed and used.

5 Work Distribution

While everything was collaborative here is the breakdown:

5.1 Common parts

Core Raft Consensus Logic:

- Developed the `RaftServer` class in `raft_server.py`, managing the overall Raft consensus algorithm including leader election, state transitions, and maintaining server state.
- Implemented state transitions through methods like `become_leader` and `become_follower`.

5.2 Rishabh (2023121001)

• Key-Value Store Implementation:

- Developed the `KeyValueStore` class in `raft_server.py`, including the `get` and `set` methods to handle client requests for data retrieval and modification.

- Ensured thread-safety within the key-value store operations using appropriate locking mechanisms.
- **Log Replication:**
 - Implemented log replication logic within the `RaftServer` class, particularly in the `AppendEntries` method to ensure that log entries are consistently replicated across all follower nodes.
 - Managed the `match_index` and `next_index` for each peer to track replication progress.
- **Snapshots and Log Compaction:**
 - Designed and implemented the `Snapshot` class to represent snapshots of the state machine.
 - Developed methods such as `create_snapshot` and `apply_snapshot` in `RaftServer` to handle log compaction and apply snapshots to the key-value store.
 - Created the `verify_snapshot.py` script to validate the integrity and correctness of snapshots.

5.3 Monish (2022111003)

- **Node Communication and RPC Handling:**
 - Designed and implemented the `Node` class in `node.py` to manage node instances, handle incoming RPCs, and facilitate communication between nodes.
 - Developed the `rpc.py` module, including the `rpc_call` decorator to streamline the registration and handling of RPC methods.
 - Implemented RPC handlers such as `RequestVote` and `AppendEntries` to process incoming consensus messages.
- **Server Initialization and Command-Line Interface:**
 - Developed the `server.py` script to serve as the entry point for starting Raft nodes, handling argument parsing, logging setup, and initializing server components.
 - Implemented the command-line interface within `server.py` to allow user interactions such as sending messages, setting and getting key-value pairs, and manually promoting a node to leader.

- **Election and Heartbeat Mechanisms:**
 - Implemented election timers and heartbeat intervals within the `RaftServer` class to manage leader elections and maintain leadership through regular heartbeats.
 - Managed election timeouts and handled the logic to reset or trigger elections based on node states and message receptions.
- **Log and State Management:**
 - Managed the persistent and volatile state of the server through the `ServerState`, `PersistentServerState`, and `VolatileLeaderState` data classes.
 - Implemented methods to save and load server state to and from durable storage, ensuring persistence across restarts.

6 Bibliography

Lamport, Leslie (1998). *The Part-Time Parliament*, ACM Trans. Comput. Syst..

Ongaro, Diego and Ousterhout, John (2014). *In Search of an Understandable Consensus Algorithm*, USENIX Association.